

SmartAdmin 《Java规范 V3.0》

阅读须知

在阅读《Java规范》之前，请一定先阅读上一篇《高质量代码思想 V3.0》!!!，[前往阅读](#)

一、Java 项目规范

1.1、Java 项目命名规范

全部采用小写方式，以中划线分隔。

```
1 正例: `mall-management-system / order-service-client / user-api`  
2  
3 反例: `mall_management-system / mallManagementSystem / orderServiceClient`
```

java

1.2、方法参数规范

无论是 `controller`, `service`, `manager`, `dao` 亦或是**其他 class** 的代码，每个方法最多 `5` 个参数，如果超出 `5` 个参数的话，要封装成 `javabean` 对象。

- 方便他人调用，降低出错几率。尤其是当参数是同一种类型，仅仅依靠顺序区分，稍有不慎便是灾难性后果，而且排查起来也极其恶心。
- 保持代码整洁、清晰度。当一个个方法里充斥着堆堆参数的时候，再坚强的人，也会身心疲惫。

反例：

java

```
1  /**
2   * 使用证书加密数据工具方法
3   *
4   * @param param 参数
5   * @param password 加密密码
6   * @param priCert 私钥
7   * @param pubCert 公钥
8   * @param username 用户名
9   * @param ip ip地址
10  * @param userAgent 用户特征
11  * @return 返回加密后的字符串
12  */
13  public String signEnvelop(JdRequestParam param, String password, String priCert,
14
15  }
```

1.3、代码目录结构

统一的目录结构是所有项目的基础。

	xml
1	src
2	-- common
3	-- config
4	-- constant
5	-- handler
6	-- interceptor
7	-- listener
8	-- module
9	-- -- employee
10	-- -- role
11	-- -- login
12	-- third
13	-- util
14	-- Application.java

源码目录
各个项目的通用类库
项目的配置信息
全局公共常量
全局处理器
全局连接器
全局监听器
各个业务(方便将来拆成微服务)
员工模块
角色模块
登录模块
三方服务, 比如redis, oss, 微信sdk等等
全局工具类
启动类

1.4、common 目录规范

common 目录用于存放各个项目通用的项目, 但是又可以依照项目进行特定的修改。

```
1  src 源码目录
2  |-- common 各个项目的通用类库
3  |-- |--- anno          通用注解，比如权限，登录等等
4  |-- |--- constant      通用常量，比如 ResponseCodeConst
5  |-- |--- domain        全局的 javabean，比如 BaseEntity,PageParamDTO 等
6  |-- |--- exception     全局异常，如 BusinessException
7  |-- |--- json          json 类库，如 LongJsonDeserializer, LongJsonSerializer
8  |-- |--- swagger       swagger 文档
9  |-- |--- validator     适合各个项目的通用 validator，如 CheckEnum, CheckBigDecimal
```

1.5、module 目录规范

module 目录里写项目的各个业务，每个业务一个独立的顶级文件夹，在文件里进行 mvc 的相关划分。其中，domain 包里存放 entity, dto, vo, bo 等 javabean 对象

```
1  src
2  |-- module              所有业务模块
3  |-- |-- role           角色模块
4  |-- |-- |--RoleController.java      controller
5  |-- |-- |--RoleConst.java          role相关的常量
6  |-- |-- |--RoleService.java        service
7  |-- |-- |--RoleDao.java            dao
8  |-- |-- |--domain                 domain
9  |-- |-- |-- |-- RoleEntity.java    表对应实体
10 |-- |-- |-- |-- RoleForm.java      请求Form对象
11 |-- |-- |-- |-- RoleVO.java        返回对象
12 |-- |-- employee          员工模块
13 |-- |-- login            登录模块
14 |-- |-- email            邮件模块
15 |-- |-- ....             其他
```

二、MVC 规范

2.1、整体分层

- controller 层

- service 层
- manager 层
- dao 层

2.2、controller 层规范

1) 只允许在 method 上添加 RequestMapping 注解

只允许在 method 上添加 `RequestMapping` 注解，不允许加在 `class` 上（为了方便的查找 url，放到 class 上 url 不能一次性查找出来）

正例：

java

```
1  @RestController
2  public class DepartmentController {
3
4      @GetMapping("/department/list")
5      public ResponseDTO<List<DepartmentVO>> listDepartment() {
6          return departmentService.listDepartment();
7      }
```

反例：

java

```
1  @RequestMapping ("/department")
2  public class DepartmentController {
3
4      @GetMapping("/list")
5      public ResponseDTO<List<DepartmentVO>> listDepartment() {
6          return departmentService.listDepartment();
7      }
```

2) 不推荐使用 restful 命名 url

不推荐使用 restful 命名 url，只能使用 `get/post` 方法。url 命名遵循：`/业务模块/子模块/动作`；

其中 `业务模块和子模块` 使用 名字，`动作` 使用动词；原因：

虽然 restful 大法好，但是有时并不能一眼根据 url 看出来是什么操作，所以选择了后者，这个没有对与错，只有哪个更适合团队

正例:

java

```
1 GET /department/get/{id} 查询某个部门详细信息
2 POST /department/query 复杂查询
3 POST /department/add 添加部门
4 POST /department/update 更新部门
5 GET /department/delete/{id} 删除部门
6
7 GET /department/employee/delete/{id} 删除部门员工
```

3) swagger 接口注释必须加上后端作者

每个方法必须添加 `swagger` 文档注解 `@ApiOperation` , 并填写接口描述信息, 描述最后必须加上接口的作者信息, 格式如下: `@author 卓大`

正例:

java

```
1 @ApiOperation("更新部门信息 @author 卓大")
2 @PostMapping("/department/update")
3 public ResponseDTO<String> updateDepartment(@Valid @RequestBody DepartmentUp
4     return departmentService.updateDepartment(departmentUpdateForm);
5 }
```

4) controller 每个方法要保持简洁

controller 在mvc中负责协同和委派业务, 充当路由的角色, 所以要保持代码少量和清晰, 要做到如下要求:

- 不做任何的业务逻辑操作
- 不做任何的参数、业务校验, 参数校验只允许使用@Valid 注解做简单的校验
- 不做任何的数据组合、拼装、赋值等操作

正例:

java

```
1 @ApiOperation("添加部门 @author 卓大")
2 @PostMapping("/department/add")
3 public ResponseDTO<String> addDepartment(@Valid @RequestBody DepartmentAddFo
4
5
```

```
        return departmentService.addDepartment(departmentAddForm);  
    }  
}
```

5) 只能在 controller 层获取当前请求用户

只能在 `controller` 层获取当前请求用户，并传递给 `service` 层。

因为获取当前请求用户是从 `ThreadLocal` 里获取取的，在 `service`、`manager`、`dao` 层极有可能是其他非 `request` 线程调用，会出现 `null` 的情况，尽量避免

java

```
1      @ApiOperation("添加员工 @author 卓大")  
2      @PostMapping("/employee/add")  
3      public ResponseDTO<String> addEmployee(@Valid @RequestBody EmployeeAddForm e  
4          RequestUser requestUser = SmartRequestUtil.getRequestUser();  
5          return employeeService.addEmployee(employeeAddForm, requestUser);  
6      }
```

2.3、 service 层规范

1) 合理拆分 service 业务

不建议service文件行数太大，如果业务较大，请拆分为多个 service；

如订单业务,所有业务都写到 `OrderService` 中会导致文件过大，故需要进行拆分如下：

- `OrderQueryService` 订单查询业务
- `OrderCreateService` 订单新建业务
- `OrderDeliverService` 订单发货业务
- `OrderValidatorService` 订单验证业务

2) 谨慎使用 @Transactional 事务注解

谨慎使用 `@Transactional` 事务注解的使用，不要简单对 `service` 的方法添加个 `@Transactional` 注解就觉得万事大吉了。

应当合并对数据库的操作，尽量减少添加了 `@Transactional` 方法内的业务逻辑。

`@Transactional` 注解内的 `rollbackFor` 值必须使用异常 `Exception.class`

对于@Transactional 注解，当 spring 遇到该注解时，会自动从数据库连接池中获取 connection，并开启事务然后绑定到 ThreadLocal 上，如果业务并没有进入到最终的操作数据库环节，那么就没有必要获取连接并开启事务，应该直接将 connection 返回给数据库连接池，供其他使用（比较难以讲解清楚，如果不懂的话就主动去问）。

反例：

java

```
1      @Transactional(rollbackFor = Exception.class)
2      public ResponseDTO<String> upOrDown(Long departmentId, Long swapId) {
3          // 验证 1
4          DepartmentEntity departmentEntity = departmentDao.selectById(departmentId);
5          if (departmentEntity == null) {
6              return ResponseDTO.wrap(DepartmentResponseCodeConst.NOT_EXISTS);
7          }
8          // 验证 2
9          DepartmentEntity swapEntity = departmentDao.selectById(swapId);
10         if (swapEntity == null) {
11             return ResponseDTO.wrap(DepartmentResponseCodeConst.NOT_EXISTS);
12         }
13         // 验证 3
14         Long count = employeeDao.countByDepartmentId(departmentId);
15         if (count != null && count > 0) {
16             return ResponseDTO.wrap(DepartmentResponseCodeConst.EXIST_EMPLOYEE);
17         }
18         // 操作数据库 4
19         Long departmentSort = departmentEntity.getSort();
20         departmentEntity.setSort(swapEntity.getSort());
21         departmentDao.updateById(departmentEntity);
22         swapEntity.setSort(departmentSort);
23         departmentDao.updateById(swapEntity);
24         return ResponseDTO.succ();
25     }
```

以上代码前三步都是使用 connection 进行验证操作，由于方法上有@Transactional 注解，所以这三个验证都是使用的同一个 connection。

若对于复杂业务、复杂的验证逻辑，会导致整个验证过程始终占用该 connection 连接，占用时间可能会很长，直至方法结束，connection 才会交还给数据库连接池。

对于复杂业务的不可预计的情况，长时间占用同一个 connection 连接不是好的事情，应该尽量缩短占用时间。

正例:

java

```
1      DepartmentService.java
2
3      public ResponseDTO<String> upOrDown(Long departmentId, Long swapId) {
4          DepartmentEntity departmentEntity = departmentDao.selectById(departmentId);
5          if (departmentEntity == null) {
6              return ResponseDTO.wrap(DepartmentResponseCodeConst.NOT_EXISTS);
7          }
8          DepartmentEntity swapEntity = departmentDao.selectById(swapId);
9          if (swapEntity == null) {
10             return ResponseDTO.wrap(DepartmentResponseCodeConst.NOT_EXISTS);
11         }
12         Long count = employeeDao.countByDepartmentId(departmentId);
13         if (count != null && count > 0) {
14             return ResponseDTO.wrap(DepartmentResponseCodeConst.EXIST_EMPLOYEE);
15         }
16         departmentManager.upOrDown(departmentEntity, swapEntity);
17         return ResponseDTO.succ();
18     }
19
20
21     DepartmentManager.java
22
23     @Transactional(rollbackFor = Throwable.class)
24     public void upOrDown(DepartmentEntity departmentEntity, DepartmentEntity swapEntity) {
25         Long departmentSort = departmentEntity.getSort();
26         departmentEntity.setSort(swapEntity.getSort());
27         departmentDao.updateById(departmentEntity);
28         swapEntity.setSort(departmentSort);
29         departmentDao.updateById(swapEntity);
30     }
```

将数据在 service 层准备好，然后传递给 manager 层，由 manager 层添加@Transactional 进行数据库操作。

以上是使用 `manager` 去处理解决的，其实也可以是使用spring的 `TransactionTemplate` 事务模板解决

3) 需要注意的是：注解 `@Transactional` 事务在类的内部方法调用是不会生效的

反例：如果发生异常，saveData 方法上的事务注解并不会起作用

java

```
1  @Service
2  public class OrderService{
3
4      public void createOrder(OrderAddForm addForm){
5          this.saveData(addForm);
6      }
7
8      @Transactional(rollbackFor = Exception.class)
9      public void saveData(OrderAddForm addForm){
10         orderDao.insert(addForm);
11     }
12 }
```

Spring 采用动态代理(AOP)实现对 bean 的管理和切片，它为每个 class 生成一个代理对象。只有在代理对象之间进行调用时，可以触发切面逻辑。而在同一个 class 中，方法 A 调用方法 B，调用的是原对象的方法，而不通过代理对象。所以 Spring 无法拦截到这次调用，也就无法通过注解保证事务了。简单来说，在同一个类中的方法调用，不会被方法拦截器拦截到，因此事务不会起作用。

解决方案：

1. 可以将方法放入另一个类，如新增 `manager` 层，通过 spring 注入，这样符合了在对象之间调用的条件。
2. 启动类添加 `@EnableAspectJAutoProxy(exposeProxy = true)`，方法内使用 `AopContext.currentProxy()` 获得代理类，使用事务。

java

```
1  SpringBootApplication.java
2
3  @EnableAspectJAutoProxy(exposeProxy = true)
4  @SpringBootApplication
5  public class SpringBootApplication {}
6
7  OrderService.java
8
9  public void createOrder(OrderCreatedTO createDTO){
10     OrderService orderService = (OrderService)AopContext.currentProxy();
11     orderService.saveData(createDTO);
12 }
```

2.4、manager 层规范

manager 层的作用(引自《阿里 java 手册》):

- 对第三方平台封装的层，预处理返回结果及转化异常信息；
- 对 Service 层通用能力的下沉，如缓存方案、中间件通用处理；
- 与 DAO 层交互，对多个 DAO 的组合复用。

2.5、dao 层规范

1) 持久层框架选择

优先使用 mybatis-plus 框架。如果需要多个数据源操作的，可以选择使用 实验室的 [SmartDb](#) 框架。

2) 使用mybatis-plus的要求

- 所有 Dao 继承自 BaseMapper
- 禁止使用 Mybatis-plus 的 Wrapper 条件构建器；原因：1、SQL无法复用；2、排查如慢SQL的时候无法以搜索SQL的形式快速定位到代码；
- 禁止直接在 mybatis xml 中写死常量，应从 dao 中传入到 xml 中

正例： NoticeDao.java 常量在参数中传入到xml

java

```
1 public interface NoticeDao{
2
3     Integer noticeCount(@Param("sendStatus") Integer sendStatus);
4
5 }
```

NoticeMapper.xml

xml

```
1 <select id="noticeCount" resultType="integer">
2     select
3     count(1)
4     from t_notice
5     where
6     send_status = #{sendStatus}
7 </select>
```

反例：常量直接写死到 xml 中

java

```
1 public interface NoticeDao{
2
3     Integer noticeCount();
4
5 }
```

NoticeMapper.xml

xml

```
1 <select id="noticeCount" resultType="integer">
2     select
3     count(1)
4     from t_notice
5     where
6     send_status = 0
7 </select>
```

4) 连接join写法

建议在xml中的 join 关联写法使用表名的全称，而不是用别名，对于关联表太多的话，在xml格式中，其实很难记住 别名是什么意思！

反例：t_notice 别名 tn，t_employee别名 e，在xml中已经很难区分是什么意思了，索性不如使用全称

xml

```
1 <select id="queryPage" resultType="net.lab1024.sa.admin.module.business.oa.notice"
2     SELECT tn.*,
3     e.actual_name AS createUserName
4     FROM t_notice tn
5     LEFT JOIN t_employee e ON tn.create_user_id = e.employee_id
6     <where>
7         tn.deleted_flag = #{queryForm.deletedFlag}
8         <if test="queryForm.keywords != null and queryForm.keywords != ''">
9             AND (INSTR(tn.notice_title,#{queryForm.keywords}) OR INSTR(e.actual_name,#{queryForm.keywords}))
10        </if>
11        <if test="queryForm.noticeType != null">
12            AND tn.notice_type = #{queryForm.noticeType}
13        </if>
14        <if test="queryForm.noticeBelongType != null">
15            AND tn.notice_belong_type = #{queryForm.noticeBelongType}
16        </if>
17    </where>
18 </select>
```

```
16         AND tn.notice_belong_type = #{queryForm.noticeBelongType}
17     </if>
18     <if test="queryForm.startTime != null">
19         AND DATE_FORMAT(tn.publish_time, '%Y-%m-%d') >= #{queryForm.s
20     </if>
21     <if test="queryForm.endTime != null">
22         AND DATE_FORMAT(tn.publish_time, '%Y-%m-%d') <= #{queryForm.e
23     </if>
24     <if test="queryForm.disabledFlag != null">
25         AND tn.disabled_flag = #{queryForm.disabledFlag}
26     </if>
27 </where>
28 <if test="queryForm.sortItemList == null or queryForm.sortItemList.size
29     ORDER BY tn.top_flag DESC,tn.publish_time DESC
30 </if>
</select>
```

正确：使用全称

```
1 <select id="queryPage" resultType="net.lab1024.sa.admin.module.business.oa.n
2     SELECT t_notice.*,
3     t_employee.actual_name AS createUser_name
4     FROM t_notice
5     LEFT JOIN t_employee ON t_notice.create_user_id = t_employee.employee_i
6 <where>
7     t_notice.deleted_flag = #{queryForm.deletedFlag}
8     <if test="queryForm.keywords != null and queryForm.keywords != ''">
9         AND (INSTR(t_notice.notice_title,#{queryForm.keywords}) OR INSTR
10    </if>
11    <if test="queryForm.noticeType != null">
12        AND t_notice.notice_type = #{queryForm.noticeType}
13    </if>
14    <if test="queryForm.noticeBelongType != null">
15        AND t_notice.notice_belong_type = #{queryForm.noticeBelongType}
16    </if>
17    <if test="queryForm.startTime != null">
18        AND DATE_FORMAT(t_notice.publish_time, '%Y-%m-%d') >= #{query
19    </if>
20    <if test="queryForm.endTime != null">
21        AND DATE_FORMAT(t_notice.publish_time, '%Y-%m-%d') <= #{query
22    </if>
23    <if test="queryForm.disabledFlag != null">
```

```
24         AND t_notice.disabled_flag = #{queryForm.disabledFlag}
25     </if>
26 </where>
27     <if test="queryForm.sortItemList == null or queryForm.sortItemList.size
28         ORDER BY t_notice.top_flag DESC,t_notice.publish_time DESC
29     </if>
30 </select>
```

2.6、javabean 命名规范

1) javabean 的整体要求:

- 不得有任何的业务逻辑或者计算
- 基本数据类型必须使用包装类型 (`Integer`, `Double`, `Boolean` 等)
- 不允许有任何的默认值
- 每个属性必须添加注释, 并且必须使用多行注释。
- 必须使用 `lombok` 简化 `getter/setter` 方法
- 建议对象使用 `lombok` 的 `@Builder`, `@NoArgsConstructor`, 同时使用这两个注解, 简化对象构造方法以及 `set` 方法。

2) javabean 名字划分:

- `XxxEntity` 数据库持久对象
- `XxxVO` 返回前端对象 (一些大厂用 `Resp`结尾, 比如 `XxxxResp`)
- `XxxForm` 前端请求对象 (一些大厂用 `Req`结尾, 比如 `XxxxReq`)
- `XxxDTO` 数据传输对象
- `XxxBO` 内部处理对象

3) 数据对象; `XxxxEntity`, 要求:

- 以 `Entity` 为结尾 (阿里是为 `DO` 为结尾)
- `Xxxx` 与数据库表名保持一致
- 类中字段要与数据库字段保持一致, 不能缺失或者多余
- 类中的每个字段添加注释, 并与数据库注释保持一致
- 不允许有组合
- 项目内的日期类型必须统一, 使用 `java.time.LocalDateTime` 或者 `java.time.LocalDate`

4) 请求对象; `XxxxForm` , 要求:

- 不可以继承自 `Entity`
- `Form` 可以继承、组合其他 `DTO`, `VO`, `BO` 等对象
- `Form` 只能用于前端、RPC 的请求参数

3) 返回对象; `XxxxVO` , 要求:

- 不可继承自 `Entity`
- `VO` 可以继承、组合其他 `DTO`, `VO`, `BO` 等对象
- `VO` 只能用于返回前端、rpc 的业务数据封装对象

4) 业务对象 `BO` , 要求:

- 不可以继承自 `Entity`
- `BO` 对象只能用于 `service`, `manager`, `dao` 层, 不得用于 `controller` 层

2.7、boolean 类型的属性命名规范

类中布尔类型的变量, 都不要加 `is`, 否则部分框架解析会引起序列化错误。反例: 定义为基本数据类型 `Boolean isDeleted`; 的属性, 它的方法也是 `isDeleted()`, RPC 在反向解析的时候, “以为”对应的属性名称是 `deleted`, 导致属性获取不到, 进而抛出异常。

这是阿里巴巴开发手册中的原文, 团队的规定是: `boolean` 类型的类属性和数据表字段都统一使用 `flag` 结尾。虽然使用 `isDeleted`, `is_deleted` 从字面语义上更直观, 但是比起可能出现的潜在错误, 这点牺牲还是值得的。

正例:

```
1      deletedFlag, deleted_flag, onlineFlag, online_flag
```

三、数据库 规范

3.1、数据库命名

全部采用小写方式, 以下划线分隔, 并且区分是什么环境的数据库

```
1  正例: smart_admin_v2_dev / smart_admin_v2_prod / smart_admin_v2_test
2
3  反例: mall_management-system / mallManagementSystem / orderServiceClient
```

3.2、表命名

全部采用小写方式，以下划线分隔，并且并且以 `t_` 开头

比如：员工表： `t_employee` 、部门表： `t_department` 、配置表： `t_config`

3.3、建表规范

若需要的话，表必备三字段： `[module]_id`, `create_time`, `update_time` ，简单表除外：如 `log` 表，不需要 `update_time` 等

- `[module]_id` 字段 Long 类型，单表自增，自增长度为 1，`module`为业务名称，如：`user_id`、`order_id` 等
- `create_time` 字段 datetime 类型，默认值 `CURRENT_TIMESTAMP`
- `update_time` 字段 datetime 类型，默认值 `CURRENT_TIMESTAMP`, On update `CURRENT_TIMESTAMP`

枚举类表字段注释需要将所有枚举含义进行注释

修改或增加字段的描述，必须要及时同步更新注释。如下表的 `sync_status` 字段 同步状态
`0` 未开始 `1`同步中 `2`同步成功 `3`失败 。

正例：

```
1  CREATE TABLE `t_change_data` (  
2      `id` BIGINT(20) UNSIGNED NOT NULL AUTO_INCREMENT,  
3      `sync_status` TINYINT(3) UNSIGNED NOT NULL DEFAULT '0' COMMENT '同步状态 0 未  
4      `sync_time` DATETIME NULL DEFAULT NULL COMMENT '同步时间',  
5      `create_time` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
6      `update_time` DATETIME NULL DEFAULT NULL ON UPDATE CURRENT_TIMESTAMP,  
7      PRIMARY KEY (`change_data_id`)  
8  )
```

反例：

```
1 CREATE TABLE `t_change_data` (  
2     `id` BIGINT(20) UNSIGNED NOT NULL AUTO_INCREMENT,  
3     `sync_status` TINYINT(3) UNSIGNED NOT NULL DEFAULT '0' COMMENT '同步状态',  
4     `sync_time` DATETIME NULL DEFAULT NULL COMMENT '同步时间',  
5     `create_time` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
6     `update_time` DATETIME NULL DEFAULT NULL ON UPDATE CURRENT_TIMESTAMP,  
7     PRIMARY KEY (`change_data_id`)  
8 )
```

3.4、索引规范

具体索引规范请参照《阿里巴巴 Java 开发手册》索引规约

联系我们

[1024创新实验室-主任：卓大](#)，混迹于各个技术圈，研究过计算机，熟悉点 java，略懂点前端。

[1024创新实验室](#)，卓大的软件公司，致力于成为中原领先、国内一流的技术团队，以AI+数字化为驱动，用技术为产业互联网提供无限可能，业务如下：

- 供应链（网络货运、大宗贸易进销存ERP、物流TMS、B2B电商、仓储WMS、AI提效等）
- 教育（就业创业大数据平台、继续教育平台、在线教育系统、题库等）
- AI+软件（软件定制外包、数据大屏、国产化改造、人员外包、技术顾问、技术培训等）
- 欢迎各类合作哦~

		 “写好代码少加班，快乐工作，热爱生活，感谢支持。” 卓大的赞赏码	 @六边形工程师 抖音号：1024lab 卓大，1024创新实验室主任：
加微信：卓大 拉你入群，一起学习	公众号：六边形工程师 分享：赚钱、代码、生活	请“1024创新实验室” 烩面里加肉 咖啡配胡辣汤,提神又饱腹	抖音：六边形工程师 直播：赚钱、代码、中医

上一篇

[Vue3规范 V3.0](#)

下一篇

[微服务与单体那些事儿](#)